

UNIVERSITÀ DEGLI STUDI DI VERONA

CORSO DI LAUREA IN INFORMATICA

Programmazione e sicurezza delle reti

Federico Brutti
federico.brutti@studenti.univr.it

C'è chi fa il panettiere e c'è chi fa l'internet service provider. - Davide Q.

Indice

1	Concetti fondamentali delle reti	3
1.1	Strumenti di analisi delle reti	3
1.2	Programmazione modello client-server	4
2	Concetti di programmazione web	7
2.1	Introduzione: HTML e CSS	7
2.2	DOM, JavaScript e struttura di web server	7
2.3	Form HTML, GET, POST	7
2.4	Prototipo completo di un web server	7
3	Configurazione ed evoluzione degli apparati di rete	8
3.1	CGI e websocket	8
3.2	Architettura orientata ai servizi	8
3.3	Architettura backend per erogazione servizi	8
3.4	Progettazione backend e cloud computing	8
3.5	Paradigma publish/subscribe, MQTT	8
4	Cybersecurity	9
4.1	Integrità e autenticazione	9
4.2	PacketTracer	9
4.3	Creazione di LAN complesse	9
4.4	Protocolli e applicativi per la sicurezza	9
4.5	Internetworking e VLAN	9
4.6	Firewall e Intrusion Detection System	9
4.7	Aspetti avanzati di rete	9

Capitolo 1

Concetti fondamentali delle reti

1.1 Strumenti di analisi delle reti

L'analisi delle reti avviene mediante il tracciamento dei pacchetti scambiati al suo interno tra gli end-host. Lo strumento utilizzato nel corso si chiama **Wireshark** e si tratta di un applicativo multiplatforma dotato di una GUI capace di eseguire gli scopi sovrastabiliti. In genere, i software analizzatori di rete come questo si "intromettono" nelle interfacce di rete normalmente gestite dai sistemi operativi; in base alla modalità di esecuzione, sarà possibile interpretare i pacchetti (modalità utente) e anche intercettarli (modalità admin/superuser).

In un setting normale del livello data-link, supponendo per esempio si stia attuando la procedura del protocollo ARP, il messaggio di richiesta è inviato in broadcast a tutti gli end-host connessi alla rete e sarà elaborato e corrisposto solo da quell'interfaccia il cui indirizzo MAC corrisponde a quello di destinazione indicato dal pacchetto. Tutte le altre interfacce vedranno il proprio sistema operativo spegnere il circuito di trasmissione della scheda di rete per ragioni di efficienza energetica e computazionale. Non invieranno quindi alcuna risposta.

Wireshark cambia questo comportamento nelle macchine in cui è eseguito in modalità admin; attiva un determinato bit per attuare la **modalità promiscua**, consentendo di intercettare anche i pacchetti non destinati all'interfaccia in esame. Naturalmente, in base al mezzo fisico utilizzato per la configurazione della rete, varia la modalità di ricezione dei pacchetti:

- **Wi-Fi:** Ogni host connesso alla rete invia pacchetti ad una determinata frequenza (2.4-5GHz) e possono essere captati da tutte le schede di rete capaci di leggerle¹.

¹Un pacchetto è leggibile quando il segnale è forte abbastanza.

- **Ethernet:** I pacchetti sono inviati mediante un cavo ethernet (doppino²) connesso ad uno switch, rendendo possibile captarli solamente se inviati in broadcast (selective flooding) da quest'ultimo o direttamente alla rete in esame.

Altri applicativi CLI per l'analisi delle reti sono i seguenti:

- **ping:** Applicativo implementato sul protocollo ICMP, il quale è usato per trasmettere informazioni relative a malfunzionamenti, come TimeExceeded o DestinationUnreachable. Prevede quindi pacchetti di diagnostica, e in questo caso specifico fa uso di **echo request** ed **echo reply**.

Lanciare un ping equivale all'invio di un echo request; l'interfaccia che lo riceve (se attiva e obbedisce al protocollo) ritorna un echo reply. Il comando calcola il roundtrip time tra le due interfacce.

- **tracert:** Basato sempre su ICMP, traccia tutta la strada che fa il pacchetto per arrivare a destinazione. Setta un TTL incrementale a partire da 1 e gli hop effettuati sono riportati grazie all'errore TTL exceeded.
- **nslookup:** Riporta l'indirizzo IP del DNS fornito come parametro.
- **whois:** Riporta nome e cognome del proprietario attuale dell'indirizzo IP o dominio indicato come parametro.
- **ifconfig:** Riporta lo stato delle interfacce di rete del proprio PC.
- **route:** Interroga la propria tabella di routing per visualizzarla ed eventualmente modificarla.

1.2 Programmazione modello client-server

Un primo approccio alla scrittura di applicazioni web richiede comprendere le due prospettive che si andranno a interfacciare; in primo luogo quella del **software**, che comprende l'interfaccia delle syscall fornite dal sistema operativo, in secondo luogo quella della **rete**, dove si lavora a livello di trasporto con i protocolli UDP e TCP facendo comunicare i processi tramite socket. Distinguiamo inoltre i seguenti concetti fondamentali:

- **Host:** Elaboratore o router le cui interfacce sono identificate mediante indirizzi IP, ai quali può essere associato anche un nome Internet.
- **Processo:** Programma in esecuzione sull'host che in un contesto di applicativi web trasmette/riceve pacchetti da/verso altri host tramite la rete. I valori identificativi spaziano nell'intervallo $[0, \dots, 65535]$.

²Ne esistono di due tipi: Unshielded twisted pair (UTP) e Shielded twisted pair (STP).

- **Applicazione:** Collaborazione tra un insieme di processi sparsi sulla rete per eseguire un compito.

Partiamo dal presupposto che lo sviluppo delle applicazioni di rete si basano su due paradigmi principali: il modello **client-server** e **publisher-subscriber**, quest'ultimo approfondito più in là nella dispensa. Se consideriamo il primo pattern abbiamo due attori fondamentali i cui panni possono essere vestiti dai vari processi: il **client**, colui che fa il primo passo e invia le richieste, e il **server**, ciò che ne risponde e si rimette in attesa di ulteriori richieste³.

Un esempio di applicazione di rete è il Web stesso, i cui client sono i web browser come Chrome o Firefox cooperanti con un server HTTP Apache, oppure più semplicemente un'applicazione di messaggistica come Telegram o Whatsapp dove gli utenti si affidano a un server apposito per l'invio e ricezione dei messaggi.

In base ai requisiti sarà necessario scegliere un protocollo per lo scambio delle sequenze di byte, per i quali abbiamo due strade principali: applicazioni **orientate al datagramma**, le quali si servono di UDP e che rendono ogni pacchetto scambiato indipendente dagli altri, e applicazioni **orientate alla connessione**, che fanno uso di TCP dando una relazione di appartenenza ai pacchetti.

Adottando un protocollo è necessario implementare correttamente le relative funzioni; nel caso di UDP abbiamo uno scambio di messaggi non affidabile e per questa ragione è utilizzato comunemente per applicazioni multimediali, mentre per TCP sarà necessario rispettare i campi per il numero di sequenza, la meccanica di acknowledge, etc. Viene usato normalmente per applicazioni di scambio dati e l'insieme dei pacchetti è determinato dalla **sessione**, la quale inizia quando si instaura la connessione e termina la sua chiusura postuma all'invio dell'ultimo pacchetto. Inoltre, nulla vieta di modificare il protocollo base come lo si preferisca, sarebbe possibile rendere UDP affidabile.

Quindi per scrivere applicazioni di rete con interfaccia socket è necessario istanziare un riferimento ad essa e attaccarcisi per usufruire dei servizi di rete. Instaurata la connessione con il relativo protocollo scelto, si potranno invocare le chiamate a funzione delle primitive per attivarla; in particolare abbiamo:

- **UDP:** *UDPSend()* per inviare un pacchetto, *UDPReceive()* per riceverlo. Quest'ultima funzione è bloccante e senza ricevere un pacchetto con send rimarrà indefinitamente in attesa.
- **TCP:** *createTCPserver()* per creare la connessione, *acceptConnection()* per accettare unilateralmente l'apertura, *closeConnection()* per chiuderla unilateralmente, *TCPSend()* per inviare pacchetti e *TCPReceive()* per riceverli. In questo contesto,

³Il client può richiedere ma anche inviare informazioni, come nel caso di SMTP. Inoltre, nulla vieta di scrivere una web app con molteplici client e server.

sia `receive` che `accept` sono bloccanti; la prima per le stesse ragioni di UDP, mentre la seconda lo è perché richiede l'apertura di una connessione.

In particolare, per quanto riguarda le applicazioni orientate alla connessione, è possibile vedere una connessione TCP come un file, rendendo quindi possibile usare il concetto di file descriptor. Inoltre, la loro garanzia di consegna è fondamentale per ulteriori protocolli che si basano su di esso, come TLS o SSH per la sicurezza dei dati.

Capitolo 2

Concetti di programmazione web

2.1 Introduzione: HTML e CSS

2.2 DOM, JavaScript e struttura di web server

2.3 Form HTML, GET, POST

2.4 Prototipo completo di un web server

Capitolo 3

Configurazione ed evoluzione degli apparati di rete

3.1 CGI e websocket

3.2 Architettura orientata ai servizi

WebServices, WebServices REST

3.3 Architettura backend per erogazione servizi

3.4 Progettazione backend e cloud computing

3.5 Paradigma publish/subscribe, MQTT

Capitolo 4

Cybersecurity

- 4.1 Integrità e autenticazione
- 4.2 PacketTracer
- 4.3 Creazione di LAN complesse
- 4.4 Protocolli e applicativi per la sicurezza
- 4.5 Internetworking e VLAN
- 4.6 Firewall e Intrusion Detection System
- 4.7 Aspetti avanzati di rete